

Relatório de Laboratório

Laboratório de Experimentação de Software — Modelo/Template de Relatório

Curso	Engenharia de Software
Disciplina	Laboratório de Experimentação de Software
Turno / Período	Noite / 6º
Professor(a)	Danilo Maia
Laboratório	Lab02 - Assistentes de IA vs. codificação manual
Grupo (trio)	Áulus Batista · Maria Clara Gomes Silva de Oliveira · Vinícius Simões
Link do repositório / GitHub Projects	https://github.com/mariaoliveira27/LabExperimentacaoDeSoftware/tree/main
Data de entrega	23/09/2026

1. Introdução

O uso de IA para geração de código, tanto em forma de auxílio como para dependência, tem crescido muito nos últimos anos, mas o estado da qualidade do código gerado ainda é muito debatido.

Neste laboratório 02, identificamos seis Katas, exercícios da disciplina de AEDs (Algoritmos e Estrutura de Dados) para medir a qualidade do código gerado pela IA do Gemini (3.6 Flash) em relação ao código escrito manualmente pelos integrantes do grupo.

O objetivo deste trabalho é responder às 3 Research Questions (RQs) principais do enunciado:

- **RQ 01:** O uso de assistente de IA reduz o tempo necessário para resolver uma tarefa de programação?
 - **Métrica:** Tempo até passar em todos os testes de aceitação ("time-to-green").
 - **Métrica:** Trial que atinge o time-box (35 min) sem sucesso deve ser registrado como censurado em 35 min, não descartado — descartar distorce a comparação a favor do tratamento com mais falhas.
- **RQ 02:** O uso de assistente de IA reduz a quantidade de defeitos (testes que falham) no código produzido?
 - **Métrica:** Taxa de sucesso: % de testes de aceitação passando ao final do time-box.
 - **Métrica:** Nº absoluto de testes falhando ao final do tempo
- **RQ 03:** O uso de assistente de IA altera a complexidade ciclomática ou a duplicação do código produzido?
 - **Métrica:** Complexidade ciclomática média (McCabe) por método/função — via Radon (Python).
 - **Métrica:** Duplicação de código: % de linhas duplicadas.
 - **Métrica:** LOC (linhas de código) como métrica de controle.
 - **Métrica:** Índice de Manutenibilidade, composta (complexidade + LOC + volume de Halstead).

Para cada uma das RQs, foram propostas pelo grupo algumas hipóteses informais:

- **Hipóteses da RQ1:**
 - Sim, códigos e funções que levariam horas ou dias podem ser criados em minutos com a IA;
 - Não, o tempo necessário para um bom prompt para a IA gerar um bom código que ainda precisará de revisão e, ocasionalmente, ajustes manuais, será igual ou maior que o tempo gasto para escrever o código manualmente desde o início.
- **Hipóteses da RQ2:**
 - Pode variar de acordo com o nível do programador. No cenário de um programador mais experiente, é possível que reduza a quantidade de defeitos, o programador consegue guiar a IA de forma mais eficiente e saberá contornar geração de erros mais básicos; já para um programador menos experiente o efeito tende a ser o oposto, dado que ele não possui a experiência necessária

para criar prompts e distinguir o que seria um resultado realmente relevante na criação de um código com uma boa arquitetura.

- **Hipóteses da RQ3:**

- A IA atualmente ainda tem dificuldade de aplicar corretamente padrões de arquitetura, desrespeitando padrões de organização e duplicando métodos em locais diferentes.

O grupo também fez uma seleção de 6 Katas para este experimento, são elas:

- **Kata 01:**

- Is Recursivo: Validação recursiva de caracteres e tipos numéricos

- **Kata 02:**

- Algebra Booleana: Avaliação de expressões de álgebra booleana.

- **Kata 03:**

- Ciframento de César: Algoritmo de ciframento de caracteres em texto

- **Kata 04:**

- Alteração Aleatória: Substituição aleatória de caracteres em cadeias de texto.

- **Kata 05:**

- Busca em Vetor: Algoritmos de busca e localização em vetores/estruturas de dados.

- **Kata 06:**

- Cálculo de Cosseno: Cálculo e aproximação do valor de cosseno.

2. Contexto

Este laboratório é um experimento de comparação e análise de qualidade de códigos gerados por IA.

O objeto de estudo é o processo de resolução de 6 Katas desenvolvidos, que o grupo optou por realizar em Python.

Para lidar com a variação individual de habilidade, adotou-se o design *within-subject* contrabalanceado: cada membro do grupo resolveu metade dos Katas com o auxílio da Gemini 3.6 Flash e a outra metade de forma manual, limitados a um *time-box* de 35 minutos por rodada.

Como referências conceituais para a medição estática (RQ3), utilizou-se a ferramenta Radon (versão 6.0.1) para avaliação de Complexidade Ciclômática (McCabe) e Índice de Manutenibilidade em Python

3. Metodologia

3.1 Principais Desafios

- **Relembrar/Aprendizagem programação:** Para algumas Katas, como a Kata 04: Alteração Aleatória e a Kata 06: Cálculo de Cosseno, foi necessário um tempo maior manual para pesquisar fórmulas e como usar aleatoriedade em Python.
- **Identificação de Duplicação Justa:** Estabelecer um critério padronizado para identificar blocos duplicados de forma consistente tanto para códigos curtos (resoluções manuais objetivas) quanto para códigos potencialmente mais verbosos (resoluções geradas pela IA).

3.2 Tomadas de Decisão

- Decidiu-se que qualquer rodada experimental que atingisse o limite do time-box, de 35 minutos, sem alcançar sucesso nos testes de aceitação seria registrada como censurada. Descartar esses dados distorceria a comparação a favor do tratamento que apresentasse mais falhas, comprometendo a validade da RQ1;
- Seleção das métricas sugeridas no enunciado para a RQ1 “time-to-green” e “time-box (35 min)”;
- Seleção das métricas sugeridas no enunciado para a RQ2 “Taxa de sucesso” e “Nº absoluto de testes falhando”;
- Seleção das métricas sugeridas no enunciado para a RQ3 “Complexidade ciclomática média (McCabe) por método/função”, “Duplicação de código: % de linhas duplicadas” e “LOC (linhas de código) como métrica de controle”;

3.3 Etapas

Sprint	Entregas Efetivas	Responsável(is)	Issues
Lab01S01	Planejamento do experimento, seleção dos 6 Katas de AEDs, configuração do repositório e elaboração dos testes de aceitação. Fase inicial dos scripts de cronômetro, caso de teste por kata, bateria de teste	Áulus, Maria Clara, Vinícius	#30, #31, #32, #33, #34
Lab01S02	Execução das rodadas experimentais (manual vs. Gemini 3.6 Flash) controladas pelo time-box de 35 minutos. Coleta dos dados primários (tempo e falhas). Aprimoramento e junção dos scripts.	Áulus, Maria Clara, Vinícius	#37, #38, #39, #40, #41, #42

Sprint	Entregas Efetivas	Responsável(is)	Issues
Lab01S03	Extração das métricas estáticas via Radon (Complexidade Ciclômática e Índice de Manutenibilidade), cálculos de duplicação e elaboração dos gráficos finais. Construção do Dashboard.	Áulus, Maria Clara, Vinícius	#44, #45, #46
Relatório Final	Consolidação dados coletados do teste de Wilcoxon, discussão aprofundada de resultados e hipóteses, ameaças à validade e conclusão final.	Áulus, Maria Clara, Vinícius	#49

3.4 Ferramentas

Ferramenta	Uso
Python 3	Implementação dos scripts de mineração e análise.
Gemini 3.6 Flash	Assistente de Inteligência Artificial utilizado como tratamento experimental em metade das resoluções.
csv	Persistência e leitura dos resultados.
Radon (v 6.0.1)	Análise estática para extração de Complexidade Ciclômática, LOC e Índice de Manutenibilidade.
SciPy (Python)	Execução do Teste Estatístico de Wilcoxon para comparar as medianas pareadas entre os grupos (Manual vs. IA).
Canva + MakeCharts	Visualização dos resultados.
GitHub Issues + GitHub Projects v2	Gestão das tarefas, responsáveis, fluxo Kanban, WIP e snapshots.

3.5 Tabela de Métricas

RQ	Métrica	Definição Operacional	Unidade	Ferramenta / Fonte
RQ01	Time-to-green	Tempo decorrido até passar em todos os testes de aceitação (censurado em 35 min)	Minutos	Script de cronômetro (Python)

RQ02	Taxa de Sucesso	Porcentagem (%) de testes de aceitação passando ao final do time-box	Porcentagem (%)	Suíte de testes (unittest/pytest)
RQ02	Falhas Absolutas	Quantidade de testes (Nº) que falham ao final do tempo limite	Inteiro (Nº)	Suíte de testes (unittest/pytest)
RQ03	Complexidade Ciclométrica	Complexidade Ciclométrica média (McCabe) por método/função calculada via Radon	Adimensional	Radon (v 6.0.1)
RQ03	Duplicação de Código	Porcentagem (%) de linhas duplicadas no arquivo final analisado	Porcentagem (%)	Script de análise de duplicação (Python)
RQ03	Controle (LOC / MI)	Linhas de Código (LOC) e Índice de Manutenibilidade (CC + LOC + Halstead)	Adimensional	Radon (v 6.0.1)

3.6 Inovações Propostas pelo Grupo (30% da nota)

Não foi feita nenhuma proposta de inovação neste laboratório.

4. Resultados

4.1 Coleta de Dados

A execução do experimento resultou em rodadas válidas distribuídas igualmente entre codificação manual e codificação assistida pela IA. Todas as rodadas respeitaram o limite do tempo; rodadas inacabadas tiveram seu tempo censurado no teto de 35 minutos, e a quantidade de testes com falha foi computada para a análise da RQ02, garantindo a integridade do conjunto de dados.

Etapa	Quantidade	Uso no laboratório	Tratamento no relatório
S01	6 Katas	Seleção dos algoritmos, validação da lógica e criação da suíte de testes de aceitação.	Base estrutural padronizada para garantir que todos os testes fossem justos.
S02	18 Rodadas (9 Manuais, 9 com IA)	Coleta de dados primários de tempo e falhas sob a restrição do time-box de 35 minutos.	Usada para responder à RQ01 e RQ02 por meio da comparação direta de tempo e taxa de sucesso.

Etapa	Quantidade	Uso no laboratório	Tratamento no relatório
S03	18 Arquivos de Código	Análise estrutural final dos resultados gerados em cada rodada utilizando Radon, teste de Wilcoxon e cálculos de duplicação.	Usada para responder à RQ03, comparando a qualidade interna (verbosidade e complexidade) de ambos os grupos.

4.2 Visualização Gráfica

RQ01 O uso de assistente de IA reduz o tempo necessário para resolver uma tarefa de programação?

A Figura 1 demonstra a distribuição do tempo de resolução (time-to-green) para ambos os tratamentos. A mediana de tempo utilizando a IA é visivelmente inferior, aproximadamente 9 minutos, em comparação ao desenvolvimento manual, aproximadamente 12,5 minutos. No entanto, o topo da IA indica uma alta variância (comparado ao do manual), evidenciando que, em determinados cenários, o ganho de velocidade inicial é neutralizado pelo tempo gasto na elaboração do prompt e correção de erros gerados pela máquina.

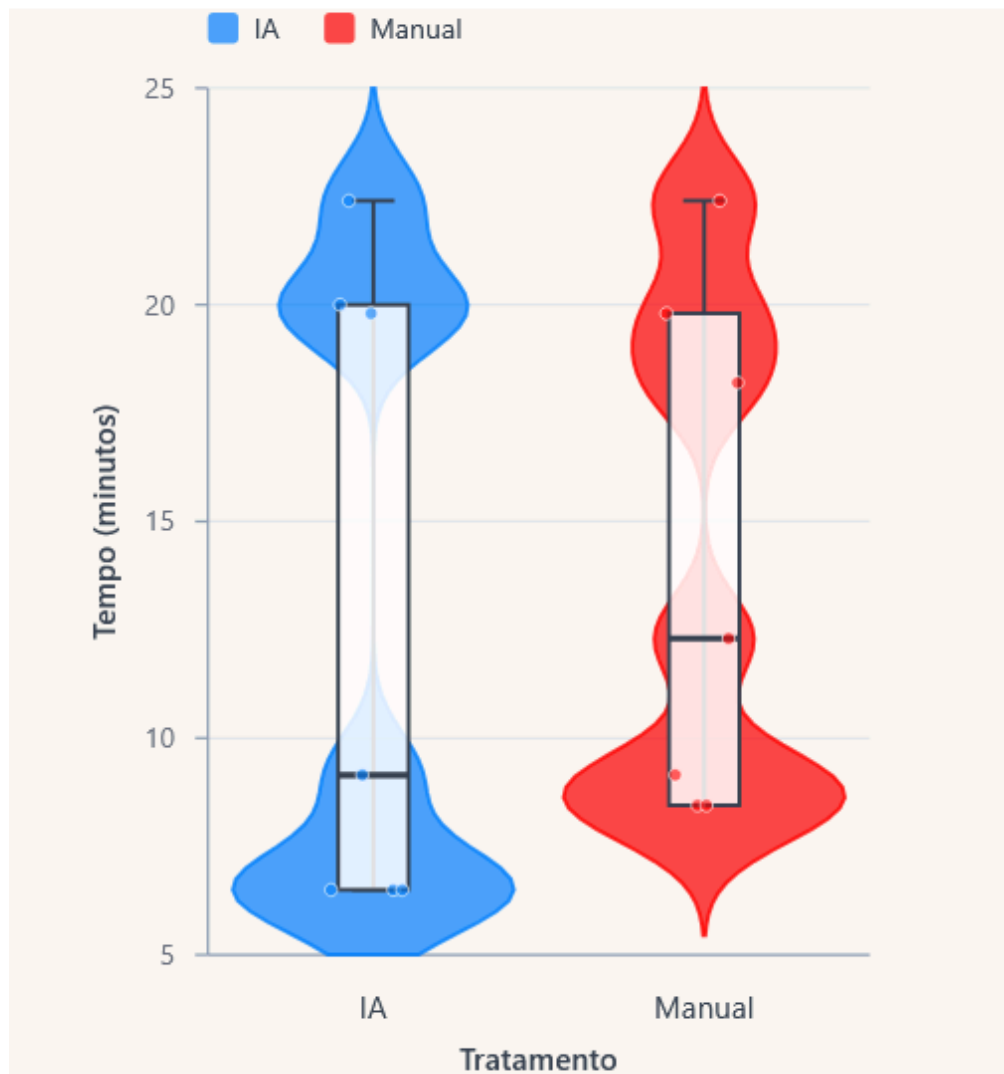


Figura 1 — RQ01: Relação de Tempo de Execução por tipo de Tratamento.

RQ02 O uso de assistente de IA reduz a quantidade de defeitos (testes que falham) no código produzido?

A análise da eficácia revela um comportamento ambivalente. A Figura 2 ilustra que, embora a IA alcance 100% de sucesso na maior parte das vezes, ela possui uma "cauda longa" que desce até 10%, demonstrando enormes falhas em certas rodadas. O desenvolvimento manual apresenta maior consistência, com sua cauda inferior parando na faixa dos 80%.

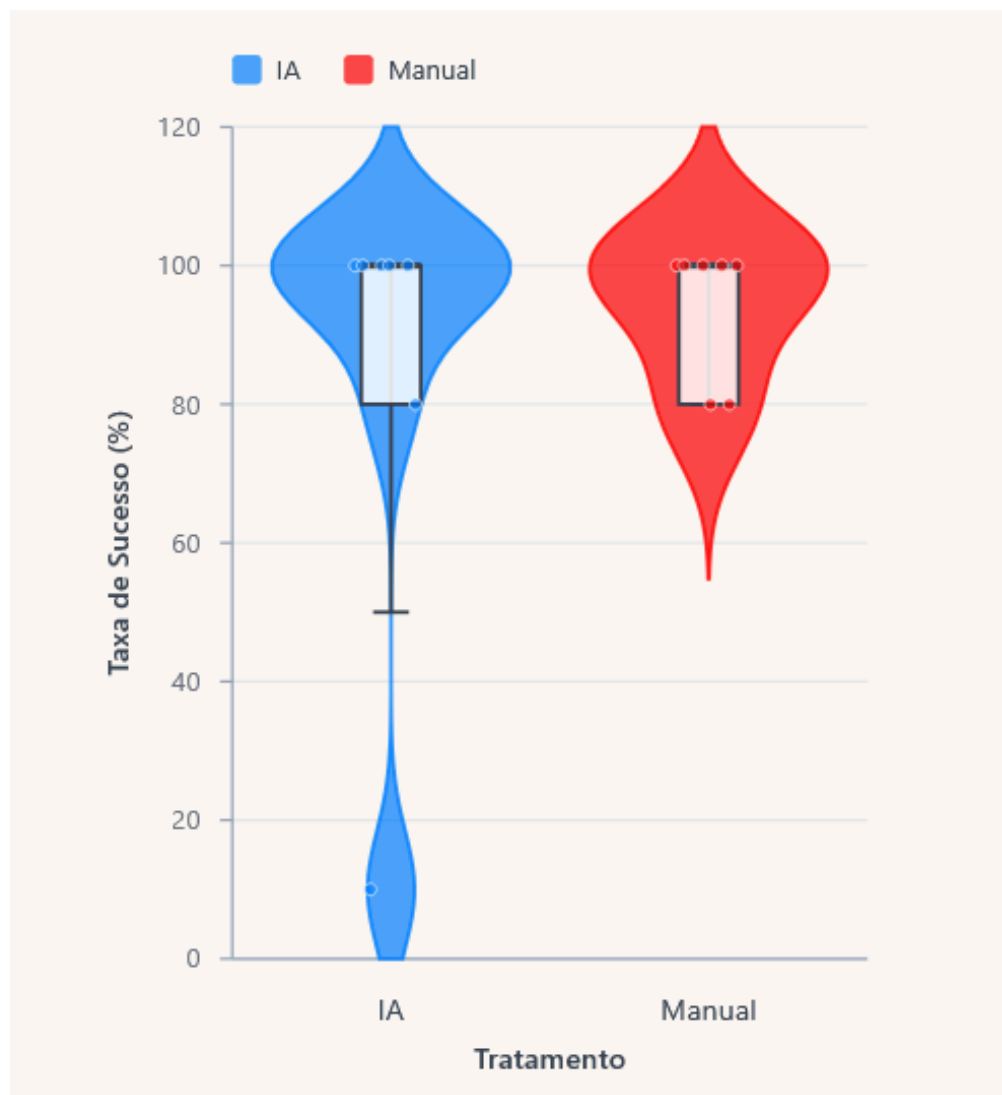


Figura 2 — RQ02: Relação de Taxa de Sucesso por tipo de Tratamento.

Essa inconsistência da IA é explicada pela Figura 3 (Média de Testes Falhados por Kata). O uso da IA não garantiu remoção de defeitos. Em problemas de lógica específica e manipulação de índices, como o Kata 04 (Alteração Aleatória) e o Kata 05 (Busca em Vetor), a IA gerou altos picos de falha (médias de 4,5 e 5,5 defeitos), enquanto a abordagem manual obteve sucesso perfeito (0 falhas). Inversamente, no Kata 06 (Cálculo de Cosseno), que exige base matemática prévia, o manual falhou (5,0), enquanto a IA obteve sucesso absoluto.

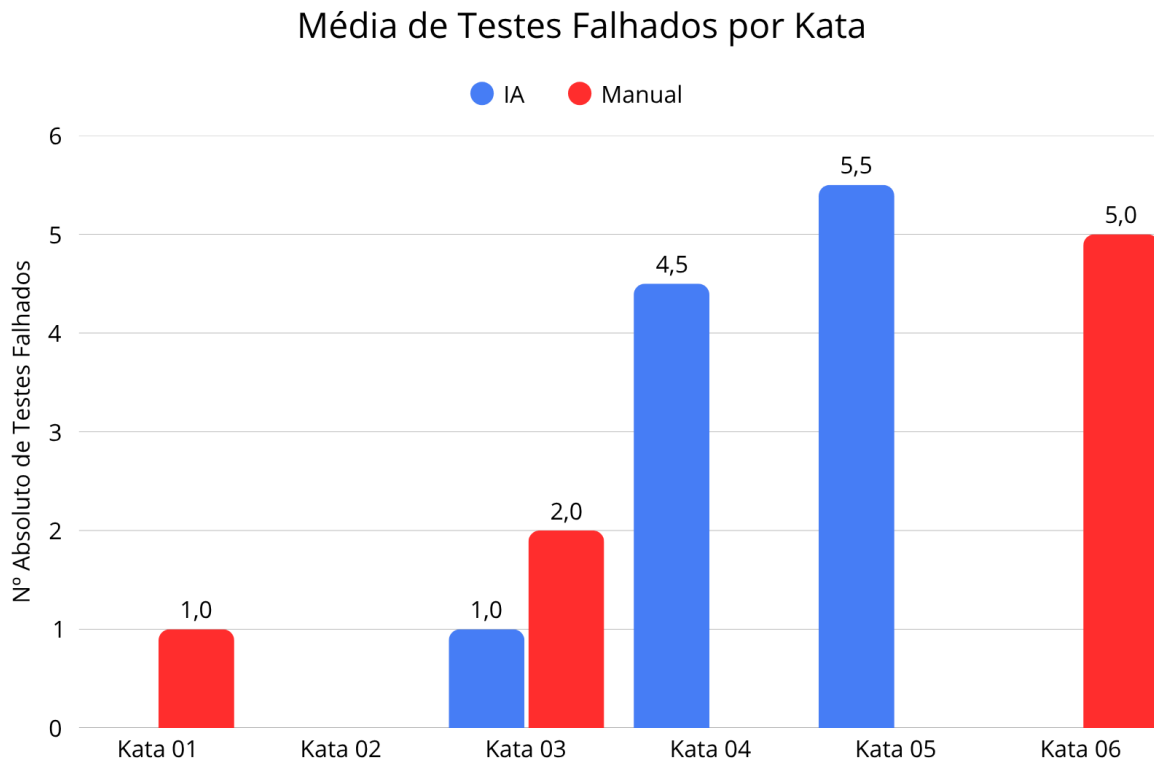
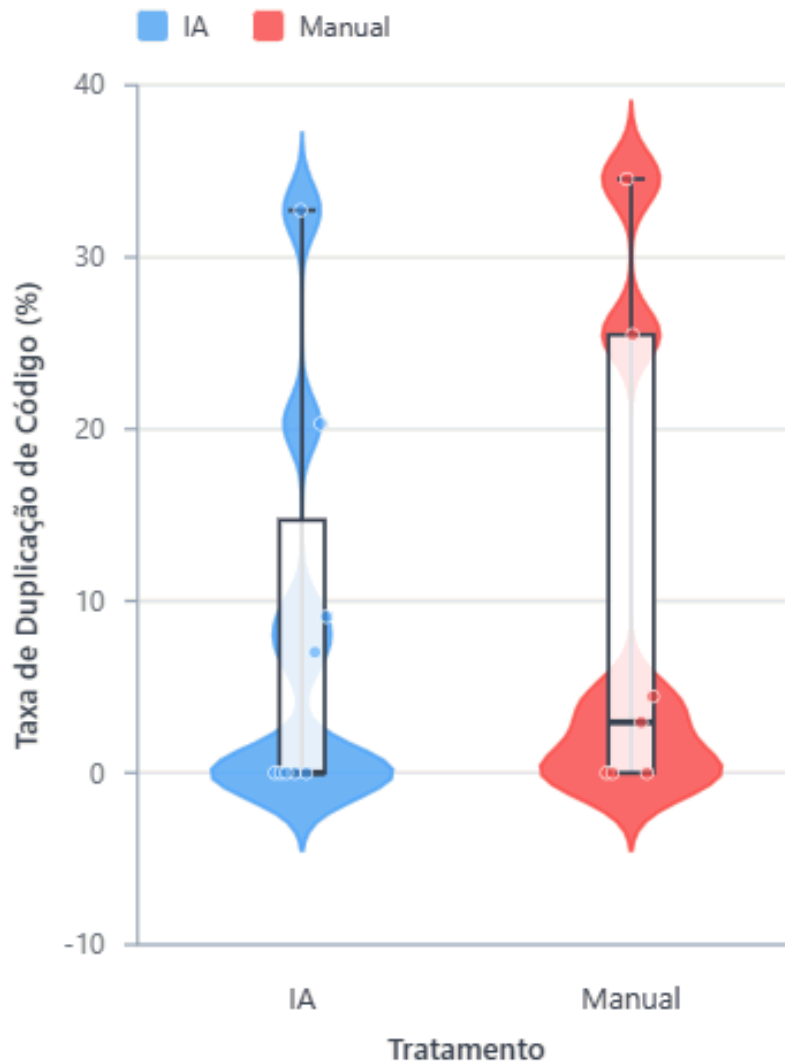


Figura 3 — RQ02: Relação de Média de Testes Falhados por tipo de Tratamento.

RQ03 O uso de assistente de IA altera a complexidade ciclomática ou a duplicação do código produzido?

Para avaliar a qualidade interna do código, utilizaram-se três visões analíticas. A Figura 4 revela que, contrariando hipótese inicial, a taxa de duplicação do código gerado pela IA foi ligeiramente menor ou equivalente à do código manual, concentrando sua densidade próxima a



0%.

Figura 4 — RQ03: Relação da Taxa de Duplicação por Kata ao tipo de Tratamento.

A Figura 5 explora a relação entre Complexidade Ciclômática (Max CC) e o Tamanho do Código (LOC). O gráfico de dispersão demonstra uma correlação linear natural para ambos os tratamentos: códigos maiores tendem a ser mais complexos. A IA gerou algumas soluções levemente mais extensas para atingir a mesma complexidade das soluções manuais, mas não demonstrou um desvio alto.

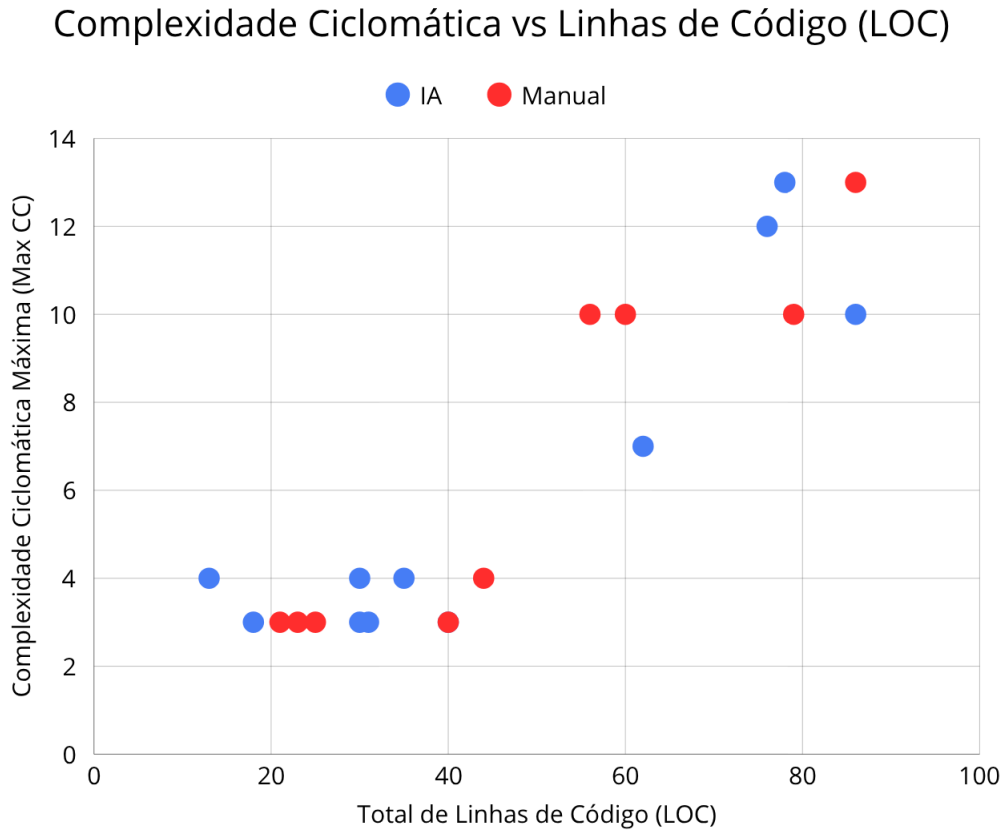


Figura 5 — RQ03: Dispersão de CC por LOC.

A Figura 6 analisa o Índice de Manutenibilidade (MI). A solução manual manteve uma mediana discretamente superior (próxima a 61) e mais agrupada. A IA apresentou uma mediana inferior (próxima a 54) e maior dispersão de resultados. Apesar dessa leve queda, os valores da IA permanecem em faixas aceitáveis, indicando códigos que ainda são passíveis de manutenção.

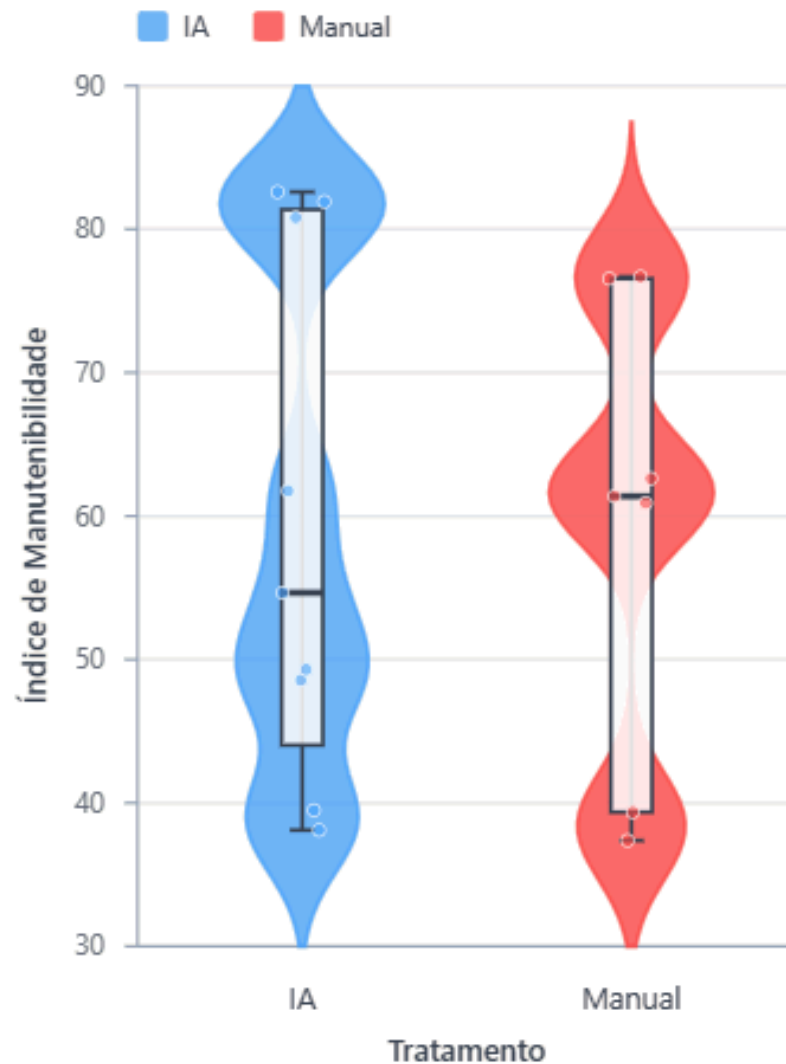


Figura 6 — RQ03: Relação do Índice de Manutenibilidade ao tipo de Tratamento.

4.3 Discussão

RQ01 (Tempo): Hipóteses Informais Confirmadas. Como teorizado, a IA tem a capacidade de gerar blocos de código em minutos, o que puxou a mediana geral de tempo para baixo (Figura 1). Contudo, a segunda parte da hipótese também se provou verdadeira: o tempo necessário para escrever bons prompts, revisar lógicas falhas e realizar ajustes manuais faz com que certas resoluções com IA igualem ou superem o tempo gasto na resolução manual. O ganho de tempo existe, mas depende fortemente do tipo do problema fornecido à IA.

RQ02 (Defeitos): Hipóteses Informais Confirmada. A IA não age como um redutor absoluto de bugs; seu desempenho variou drasticamente conforme a kata e a capacidade do programador em validar o que foi gerado. Como visto na Figura 3, a IA falhou gravemente no Kata 05 (Busca em Vetor), entregando código estruturalmente extremamente falho, que um programador precisaria refatorar por completo. Por outro lado, a IA foi capaz de solucionar com mais precisão o problema matemático no Kata 06 (Cálculo de Cosseno), mitigando falhas que ocorreram na rodada manual. Isso demonstra que a IA elimina erros de sintaxe e conhecimento de base, mas produz códigos enganosos e falhos se não for bem guiada e revisada.

RQ03 Hipótese Informal Parcialmente Refutada. Os dados coletados demonstraram que a IA não gerou mais código duplicado que a abordagem manual (Figura 4). Além disso, a complexidade ciclomática escalou de forma idêntica ao manual em proporção ao tamanho do problema (Figura 5). A única métrica que sustentou sutilmente a hipótese inicial foi o Índice de Manutenibilidade (Figura 6), que se mostrou ligeiramente inferior no tratamento com IA, evidenciando uma propensão a soluções mais verbosas em comparação às soluções manuais mais curtas.

As principais ameaças à validade são: o tamanho da amostra (18 rodadas totais) e o nível de dificuldade escolhido para as Katas, se fosse Katas mais complexas o resultado poderia se mostrar diferente do que encontramos no experimento.

5. Conclusão

O experimento revelou que a introdução de assistentes de IA generativa (Gemini 3.6 Flash) no desenvolvimento de algoritmos gera impactos contrastantes e dependentes do contexto.

Para a RQ01, verificou-se que a IA reduz o tempo bruto de geração de código, porém a economia de tempo pode sim ser contrabalanceada pelo esforço necessário de revisão e iteração de prompts.

Em relação à RQ02, a IA atua de forma ambivalente: elimina em grande parte os defeitos de sintaxe e domínio da linguagem, mas requer supervisão atenta (que a eficiência depende do nível de experiência do programador) para evitar o acúmulo de falhas lógicas não previstas em casos de borda.

Para a RQ03, confirmou-se que soluções automatizadas tendem a apresentar um aumento na verbosidade (LOC) e, por vezes, na complexidade ciclomática em cenários onde um desenvolvedor humano optaria por abstrações e padrões de projeto mais convencionais.

Conclui-se que, ao menos no cenário proposto, a IA atua primariamente como um acelerador de prototipagem, não substituindo a necessidade do julgamento arquitetural de um programador para a manutenção a longo prazo do software.

5. Referências

- MAIA, Danilo. Laboratório de Experimentação de Software: Enunciado Lab02 - Assistentes de IA vs. codificação manual. PUC Minas, 2026.